

of operations and `__enable_irq()` after. However, if interrupts were already disabled (for some other reason) before executing the sequence of operations, then it is incorrect to enable them afterward.

The correct approach is shown in Listing 4.7. The code must save the masking state (using `__get_PRIMASK()`) before disabling interrupts (using `__disable_irq()`). After performing the critical section operations, the saved masking state is restored (using `__set_PRIMASK()`).

```
void my_function(void) {
    uint32_t masking_state;
    // Perform non-critical processing
    // ...
    // Get current interrupt masking state
    masking_state = __get_PRIMASK();
    // Disable interrupts
    __disable_irq();
    // Perform critical section operations
    // ...
    // Restore previous interrupt masking state
    __set_PRIMASK(masking_state);
    // Perform more non-critical processing
    // ...
}
```

Listing 4.7 Correct method to prevent interrupts during critical section of code saves and restores interrupt masking state.

Software for Interrupts

When creating software for a system that uses interrupts, we design the system first and then implement it by writing the code. We design the system by identifying which interrupts to use and then deciding how to divide (partition) the work between ISRs and main-line code. We can then write the code to configure hardware to generate and handle interrupts, and finally write the code for the ISRs themselves.

Program Design

Selecting which interrupts to use is generally simple because they are peripheral-specific. Some peripherals have flexible interrupt generation capabilities that can simplify the software design significantly when used appropriately. We will examine these features in later chapters.

Determining how to structure the code in response to the interrupt depends on two major issues: How should we partition work between the ISR and the main-line code? How will the ISR communicate the remaining work to the main-line code? We discuss these next. These topics are covered further in other texts [7] [8] [9].

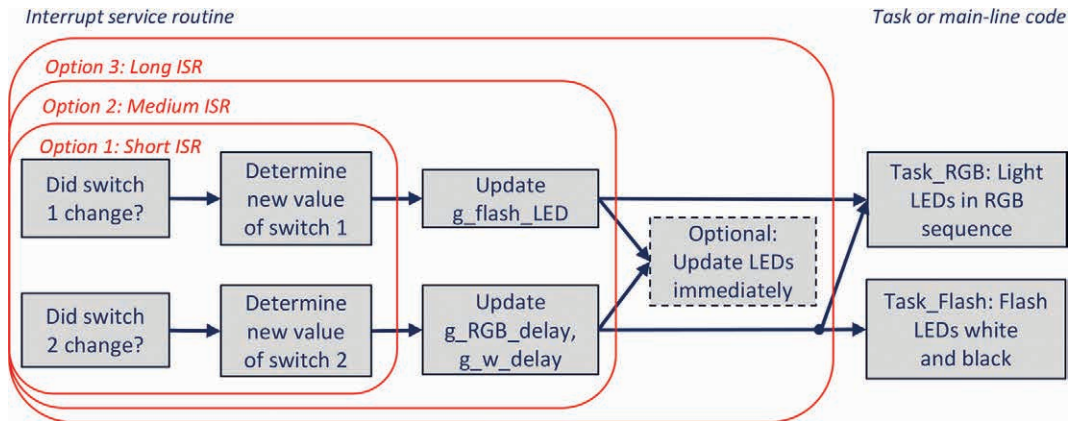


Figure 4.26 Three options for partitioning work between ISR and task code for the flashing LED example.

Partitioning

The ISR should perform only quick, urgent work related to the interrupt. Other work should be deferred to the main-line code when feasible. This keeps each ISR quick and doesn't delay other ISRs unnecessarily. Keeping ISRs short also makes the code much easier to debug.

Consider the flashing LED example from the previous chapter. Pressing or releasing a switch will trigger an interrupt. Figure 4.26 shows the work involved in response. Software must identify which switch changed, determine the new value of the switch, update the variables `g_w_delay`, `g_RGB_delay`, and `g_flash_LED`, and light the LEDs.

How much of this work should be done in the ISR, and how much should be left in the main-line code? Let's consider three options:

- **Option 1: Short ISR.** The ISR simply signals which switch changed and how (whether pressed or released). A task in the main-line program needs to update the delay and flash mode variables based on this switch information. This could be done by `Task_RGB` and `Task_Flash`, which would reduce the code's modularity. The variables could be updated by a new task, which would add the overhead of creating and running another task.
- **Option 2: Medium ISR.** The ISR directly updates the delay and flash mode variables. This is a quick, low overhead approach.
- **Option 3: Long ISR.** The ISR directly updates the delay and flash mode variables, and immediately updates the LED based on the new delay or flash mode. This approach is much more responsive than the first two, but we have a problem when the ISR completes and the previously executing task resumes. That task will light the LEDs with the wrong colors or the wrong delays. We need a way to disable or restart that task, and this is not simple.

Communication

When work is split up between the ISR and the main-line code, we need a way to communicate the intermediate results between the pieces.

In Options 2 and 3 above, the ISR directly updates the delay and flash mode variables. These can be shared variables which the ISR writes and the task code reads. This is a straightforward and simple solution.

The first option is more complex. The ISR writes to shared variables that indicate which switch changed and how. The task code reads the switch variables and then updates the delay and flash mode variables. However, how do we keep the task code from reusing the switch variables the next time it runs? Should the ISR use a flag to tell the task to run once? Or should the task erase the switch variables when it is done with them?

There are more questions to consider. What should the system do if the ISR runs more than once before the main-line task code can run? Should the task code process the data from just the first ISR instance, or just the last? If the task code must process all the data, then it must be saved somehow. For example, a system with serial communication should not lose any incoming or outgoing data. The ISR and the task code will need to coordinate on how to store the data and reuse space effectively. We will see how to do this with queues in a later chapter. Kernels and RTOSs also provide queues for application programs to use.

Example System Design

We choose the intermediate partitioning approach to create the design shown in Figure 4.26. The ISR updates the shared variables `g_w_delay`, `g_RGB_delay`, and `g_flash_LED`, which in turn are read by the tasks. We do not need to buffer or accumulate data for this example. If the

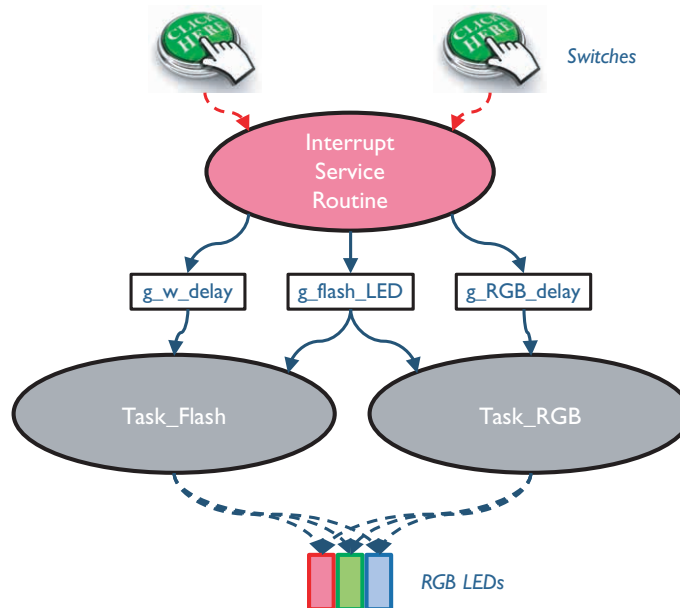


Figure 4.27 Communication and control paths among hardware and software components in example LED flashing system.

user presses a switch multiple times before a task can run, the ISR will update the shared variables multiple times. We only care about the last value of each variable, which the task uses to control the flashing.

The developer should consider the range of partitioning options to find a good balance between performance (responsiveness) and complexity. More partitioning improves responsiveness for the rest of the system, but increases the communication complexity and reduces responsiveness for the interrupt in question.

Interrupt Configuration

Listing 4.8 shows a function with the four interrupt initialization sections we examined previously. As a reminder, the code first configures the peripheral to generate interrupts. This step is peripheral-specific. In our example with the STM32 port module, we need to specify which port input bits should generate interrupts and under what circumstances. The port module is described earlier in this chapter, in the section “Peripheral Interrupt Configuration”. The two switches are connected to port C bits SW1_POS (bit 13) and SW2_POS (bit 7). If we want any input change to generate an input, we will need to set the mode of the GPIOs to generate an interrupt on the falling and the rising edge. Second, the code configures the NVIC to recognize this interrupt and with a given priority. Both switches are connected to the EXTI4_15 interrupt, so we will use the EXTI4_15_IRQn name from CMSIS-CORE. We first set the priority using a call to `NVIC_SetPriority`, then clear any pending interrupts using a call to `NVIC_ClearPendingIRQ`, and then finally enable the interrupt using `NVIC_EnableIRQ`.

```
void Init_GPIO_Switches_Interrupts(void) {
    // Enable peripheral clock of GPIOC (for switches)
    RCC->AHBENR |= RCC_AHBENR_GPIOCEN;
    // Configure PC7 and PC13 in input mode
    MODIFY_FIELD(GPIOC->MODER, GPIO_MODER_MODER7, ESF_GPIO_MODER_INPUT);
    MODIFY_FIELD(GPIOC->MODER, GPIO_MODER_MODER13, ESF_GPIO_MODER_INPUT);
    // Enable pullup on each input with 01.
    MODIFY_FIELD(GPIOC->PUPDR, GPIO_PUPDR_PUPDR7, 1);
    MODIFY_FIELD(GPIOC->PUPDR, GPIO_PUPDR_PUPDR13, 1);

    // Enable peripheral clock for SYSCFG
    RCC->APB2ENR |= RCC_APB2ENR_SYSCFGCOMPEN;
    // Select Port C for bits for SW1 and SW2
    // SW1 is at Port C bit 13
    MODIFY_FIELD(SYSCFG->EXTICR[3], SYSCFG_EXTICR4_EXTI13, 2);
    // SW2 is at Port C bit 7
    MODIFY_FIELD(SYSCFG->EXTICR[1], SYSCFG_EXTICR2_EXTI7, 2);

    // Set mask bits for inputs in EXTI_IMR
    EXTI->IMR |= MASK(SW1_POS) | MASK(SW2_POS);
    // Trigger on both rising and falling edges in EXTI_RTSR and EXTI_FTSR
    EXTI->RTSR |= MASK(SW1_POS) | MASK(SW2_POS);
    EXTI->FTSR |= MASK(SW1_POS) | MASK(SW2_POS);

    // Configure enable and mask bits for NVIC IRQ Channel for EXTI
    // Interrupt lines 7 and 13 are both serviced by EXTI4_15_IRQ
    NVIC_SetPriority(EXTI4_15_IRQn, 3);
    NVIC_ClearPendingIRQ(EXTI4_15_IRQn);
    NVIC_EnableIRQ(EXTI4_15_IRQn);

    // Optional: Configure PRIMASK in case interrupts were disabled
    __enable_irq();
}
```

Listing 4.8 Complete code to initialize PC7 and PC13 to generate interrupts on rising and falling edges.

Third, the code may ensure that interrupts are not disabled. When the processor comes out of reset, the PM bit in the CPU's PRIMASK register will be zero, so interrupts will be enabled and the PM bit will not need to be modified. However, if other code has run and set the PM bit, then it is necessary to call `__enable_irq`.

Writing ISRs in C

Let's move on to writing the actual ISR. The ISR function takes no arguments and has no return values (e.g. `void EXTI4_15_IRQHandler (void)`). An ISR must be named according to the CMSIS-CORE exception handler names shown in Listing 4.2 (e.g. `EXTI4_15_IRQHandler`, `RTC_IRQHandler`). This ensures that the software toolchain places ISR addresses in the vector table correctly. The source code for the ISR for our example system is shown in Listing 4.9.

```
volatile uint8_t g_flash_LED = 0;
volatile uint32_t g_w_delay = W_DELAY_SLOW;
volatile uint32_t g_RGB_delay = RGB_DELAY_SLOW;

void EXTI4_15_IRQHandler(void) {
    if ((EXTI->PR & SW1_POS) != 0) {
        EXTI->PR = SW1_POS; // clear pending request
        if (SWITCH_PRESSED(SW1_POS)) {
            g_flash_LED = 1; // Flash white
        } else {
            g_flash_LED = 0; // RGB sequence
        }
    }
    if ((EXTI->PR & SW2_POS) != 0) {
        EXTI->PR = SW2_POS; // clear pending request
        if (SWITCH_PRESSED(SW2_POS)) {
            // Short delays
            g_w_delay = W_DELAY_FAST;
            g_RGB_delay = RGB_DELAY_FAST;
        } else {
            // Long delays
            g_w_delay = W_DELAY_SLOW;
            g_RGB_delay = RGB_DELAY_SLOW;
        }
    }
    // Clear all other pending requests for this handler
    EXTI->PR = 0x0000fff0;
}
```

Listing 4.9 *EXTI4_15_IRQHandler* Source code for shared variables and interrupt service routine (handler).

Both switches are connected to EXTI4_15, so we name the ISR `EXTI4_15_IRQHandler` to make the corresponding vector table entry point to our ISR.

Our ISR needs to determine which switch changed, and what the switch's new value is. This ISR could be running because of a request any of twelve interrupt sources (from EXTI4 through EXTI15). The ISR reads `EXTI_PR` and tests the bit for `SW1_POS`. If it is not zero, then SW1 requested the interrupt. The ISR writes a one to that bit to tell the hardware the interrupt is being serviced, clearing the bit. The new value of the switch is determined by reading the port input data bit with the macro `SWITCH_PRESSED`. This is repeated for `SW2_POS`.

After determining which switch changed and how, the code can update the shared variables `g_flash_LED`, `g_w_delay`, and `g_RGB_delay`.

Finally, we clear all other possible pending interrupt requests for this handler by writing ones to their bits in `EXTI_PR` (positions 4 through 15). This value is binary 1111 1111 1111 0000 or hexadecimal 0xFFFF0.

Sharing Data Safely Given Preemption

Sharing data in a system with preemption introduces possible problems. Note that both interrupts and preemptive task scheduling (e.g. with a kernel) can cause preemption. Interrupts can preempt tasks. In a system with preemptive scheduling, tasks can preempt other tasks.

Volatile Data Objects

The first problem comes from the fact that shared data objects are stored in memory. When creating a function that uses the shared data, the compiler generates instructions to copy the data into a register to process it. This occurs each time the variable appears in the source code. However, if the shared data is used multiple times in the function, the compiler may optimize the code by reusing the value that was loaded the first time, rather than generating more instructions to reload it for successive uses.

```
void Task_RGB(void) {
    if (g_flash_LED == 0) { // Only run task when NOT in flash mode Control_RGB_
        LEDs(1, 0, 0);
        Delay(g_RGB_delay); // Code reads g_RGB_delay from memory into
        /* If switch 2 changes now, the ISR will run
        and update g_RGB_delay in memory */
        Control_RGB_LEDS(0, 1, 0);
        Delay(g_RGB_delay); // Error: using old value of g_RGB_delay in
        Control_RGB_LEDS(0, 0, 1);
        Delay(g_RGB_delay); // Error: using old value of g_RGB_delay in
    }
}
```

Listing 4.10 Task_RGB might reuse first value of `g_RGB_delay`.

Consider the code in Listing 4.10. The shared variable `g_RGB_delay` is used three times. The compiler generates code to load `g_RGB_delay` from memory into a register for the first call to the subroutine called `Delay`. For optimization reasons, the compiler may reuse the value that was read the first time in the second and third calls to `Delay`. Consider the case shown in the code listing, in which switch 2 changes after the first use of `g_RGB_delay`. The ISR will run and change the value of `g_RGB_delay` in memory. However, the code is still using the old value of `g_RGB_delay`, causing a program error.

In other cases the source code may specify writing the result back to memory. To speed up the program, the compiler may not save the updated values back to memory until necessary (e.g. the end of the function). Imagine that such a function is executing. It has already loaded the value and updated the value, but the code is optimized so that it does not save the updated value back to memory until the end of the function. An interrupt is requested, causing an ISR to run that changes the shared variable in memory. When the function resumes executing, it will be using the old value of the shared variable, not the new value. When the function completes, it will overwrite the new value with the updated old value, causing an error.

We call this shared data *volatile* because it can change outside a program's normal flow of control. If an ISR may change a variable used by main-line code, then that data is *volatile*. Similarly, a hardware register (e.g. a counter) that may change on its own is also *volatile*.

```
// Initially don't flash LED, just do RGB sequence
volatile uint8_t g_flash_LED = 0;
// Delay for white flash
volatile uint32_t g_w_delay = W_DELAY_SLOW;
// Delay for RGB sequence
volatile uint32_t g_RGB_delay = RGB_DELAY_SLOW;
```

Listing 4.11 Variables shared between ISR and mainline code must be defined as *volatile*.

We tell the compiler that a variable may change outside of its control by using the **volatile** keyword before the data type in the variable's definition. This forces the compiler to reload the variable from memory each time it is used in the source code. In Listing 4.11 the shared variables `g_Flash_LED`, `g_w_delay`, and `g_RGB_delay` are defined as **volatile**. This indicates to the compiler that they may change unexpectedly (e.g. an ISR may change them).

Atomic Object Access

The second problem comes from the fact that some data objects take multiple operations to modify. This means they do not have *atomic* (i.e. indivisible) access. If a function is interrupted or preempted in the middle of these operations, it is possible for the data to be corrupted.

atomic

Indivisible, cannot be interrupted or preempted

All variables located in memory are vulnerable to these data races. For example, consider the program in Listing 4.12 which lets both an ISR and the main code (in `my_bad_function`) increment the variable `g_counter`. The function `my_bad_function` will load the counter variable's value (e.g. 0) from memory into a register and increment that register's value (e.g. to 1). If the interrupt is requested before the updated value is stored back to memory, then `my_ISR` will load the old value (0) from memory into a register, increment it (1) and store it back to memory. The function `my_bad_function` will resume and write its updated value (1). The counter variable has been incremented twice, but has only gone from 0 to 1, so the data has been corrupted. This is called a *data race* situation, as it depends on the specific timing relationship between the program execution and the interrupt request. The sequence of instructions that must not be interrupted is called a *critical section*.

data race

Situation in which the ill-timed preemption of a code critical section can result in an incorrect program result

critical section

Section of code which may execute incorrectly if not executed atomically

```
volatile uint32_t g_counter;

void my_ISR(void) {
    g_counter++;
}

void my_bad_function(void) {
    gcounter++;
}

void my_good_function(void) {
    uint32_t masking_state;
    // Disable interrupts before critical section
    // Get current interrupt masking state
    masking_state = __get_PRIMASK();
    // Disable interrupts
    __disable_irq();
    // Critical section starts
    g_counter++;
    // Critical section ends
    // Restore interrupt masking state after critical section
    __set_PRIMASK(masking_state);
}
```

Listing 4.12 Disabling interrupts to make a critical section atomic.

We solve this problem by disabling preemption during the critical section. For example, we disable interrupts during the critical section in `my_good_function` as shown in Listing 4.12. As described in the section “Safely Masking Interrupts” above, we need to save the interrupt masking state, mask interrupts, and then restore the previous interrupt masking state. Preemptive kernels offer this and related methods to prevent preemption during critical sections of code.

Note that variables longer than one word will take more than one load and one store to update,¹ so they are vulnerable to an additional type of corruption: a partial update. Consider our example LED flasher system. Let’s consider changing the size of `g_RGB_delay` from one word (`uint32_t`, which is 32 bits) to two words (`uint64_t`, which is 64 bits) to allow longer time delays. The code will now need to perform two load operations to read `g_RGB_delay` from memory: `LDR R0, [g_RGB_delay]` for the low word and `LDR R1, [g_RGB_delay+4]` for the high word. Similarly, two store operations are needed to write the two words back.

If switch 2 changes while the first load instruction is executing, the data for the task will be corrupted. The CPU will complete that first load (low word) and then execute the ISR. The ISR will update the two words of `g_RGB_delay` in memory. The task code will then resume by loading R1 with the second (high) word from memory. The registers now hold corrupted data: R0 holds the old low word, and R1 holds the new high word.

Summary

In this chapter, we explored the basic organization and programmer’s model of a Cortex-M0 CPU core. We learned about general-purpose registers and special control registers in the core. We saw how memory is addressed and how a stack works. We examined the available instructions, including data movement, data processing, memory access, control flow, and miscellaneous instructions. We learned about different processor operating modes and how Thumb instructions differ from standard Arm instructions.

We then examined exceptions and interrupts. We saw the steps the CPU follows to handle exceptions: looking up a handler’s address in the vector table, entering the exception handler and then exiting it. We then covered the path that interrupts follow: from generation by a peripheral (e.g. GPIO and SYSCFG), through EXTI and the NVIC, and finally through the CPU masking hardware. Finally we discussed how to design software for interrupts: how to partition work between the ISR and main-line code, how to write software which configures the hardware to generate and recognize interrupts, and how to write the ISR. We also discussed how to handle volatile data and provide atomic object access.

Exercises

1. How does the word `0xDECD EDD1` appear in memory in a little-endian memory system as well as in a big-endian memory system? Specify the relative address for each byte.
2. Does the stack in Arm processors grow toward larger or smaller addresses?

¹ Although the Cortex-M0 supports load multiple (LDM) and store multiple (STM) instructions to process multiple registers, these instructions can be interrupted before completion.

3. Assuming that SP is 0x0000 2220 initially, what is its value after executing the instruction `PUSH {R0, R2}`?
4. Assuming that SP is 0x0000 2010 initially, what is its value after executing the instruction `POP {R0-R7, PC}`?
5. Write the Thumb code to add number five to the contents of register R6.
6. Write the Thumb code to subtract 1,000 from the contents of register R6, using R3 as a temporary register.
7. Write the Thumb code to multiply the two 32-bit values in memory at addresses 0x1234 5678 and 0x7894 5612, storing the result in address 0x2000 0010.
8. Write the Thumb code to load register R0 with the ASCII code for the letter “E” if the number in R12 is even, or “O” if it is odd.
9. Which modules generate the IRQ0, IRQ10, and IRQ30 interrupt requests, and what are their CMSIS typedef enumeration labels? Examine the interrupt vector assignments (IVA) table in the MCU reference manual [6] and the `stm32f091xc.h` file (or appropriate `device.h` file for a different MCU device).
10. We would like to configure a STM32F091RC MCU so that if interrupts IRQ0, IRQ10, and IRQ30 are requested simultaneously, the CPU responds by servicing IRQ10 first, then IRQ0, and finally IRQ30. Write the C code using CMSIS functions to configure the MCU.
11. We wish to enable IRQ13 but disable IRQ24. What values need to be loaded into which register bits, and what is the sequence of CMSIS calls to accomplish the same?
12. We wish to determine if IRQ7 has been requested. Which register and which bit will indicate this? What is the CMSIS call that will reveal the information?
13. Which register can an exception handler use to determine if it is servicing exception number 0x21? What value will the register have? What is the CMSIS interface code to read the IPSR?
14. Using EXTI_PR register we could clear the pending flags for all bits. Under what circumstances might this lead to incorrect system operation?

References

- [1] Arm Ltd., *ARMv6-M Architecture Reference Manual*, DDI 0419D, 2017.
- [2] Arm Ltd., *Cortex-M0 Devices Generic User Guide*, DUI0497A, r0p0 ed., 2009.
- [3] Arm Ltd., *Cortex-M0 Technical Reference Manual*, DDI0432C, r0p0 ed., 2009.
- [4] J. Yiu, *The Definitive Guide to the ARM Cortex-M0 and Cortex-M0+ Processors*, 2nd ed., Oxford: Newnes, 2015.
- [5] STMicroelectronics NV, *Programming Manual PM0215: STM32F0xxx Cortex-M0*, 2012.
- [6] STMicroelectronics NV, *Reference Manual RM0091: STM32F0x1/STM32F0x2/STM32F0x8*, 2017.
- [7] P. Koopman, *Better Embedded System Software*, Pittsburgh: Drumnadrochit Education LLC, 2010.
- [8] J. Ganssle, *The Art of Designing Embedded Systems*, Second ed., Elsevier Inc., 2008.
- [9] J. K. Peckol, *Embedded Systems: A Contemporary Design Tool*, John Wiley & Sons, Inc., 2008.

5